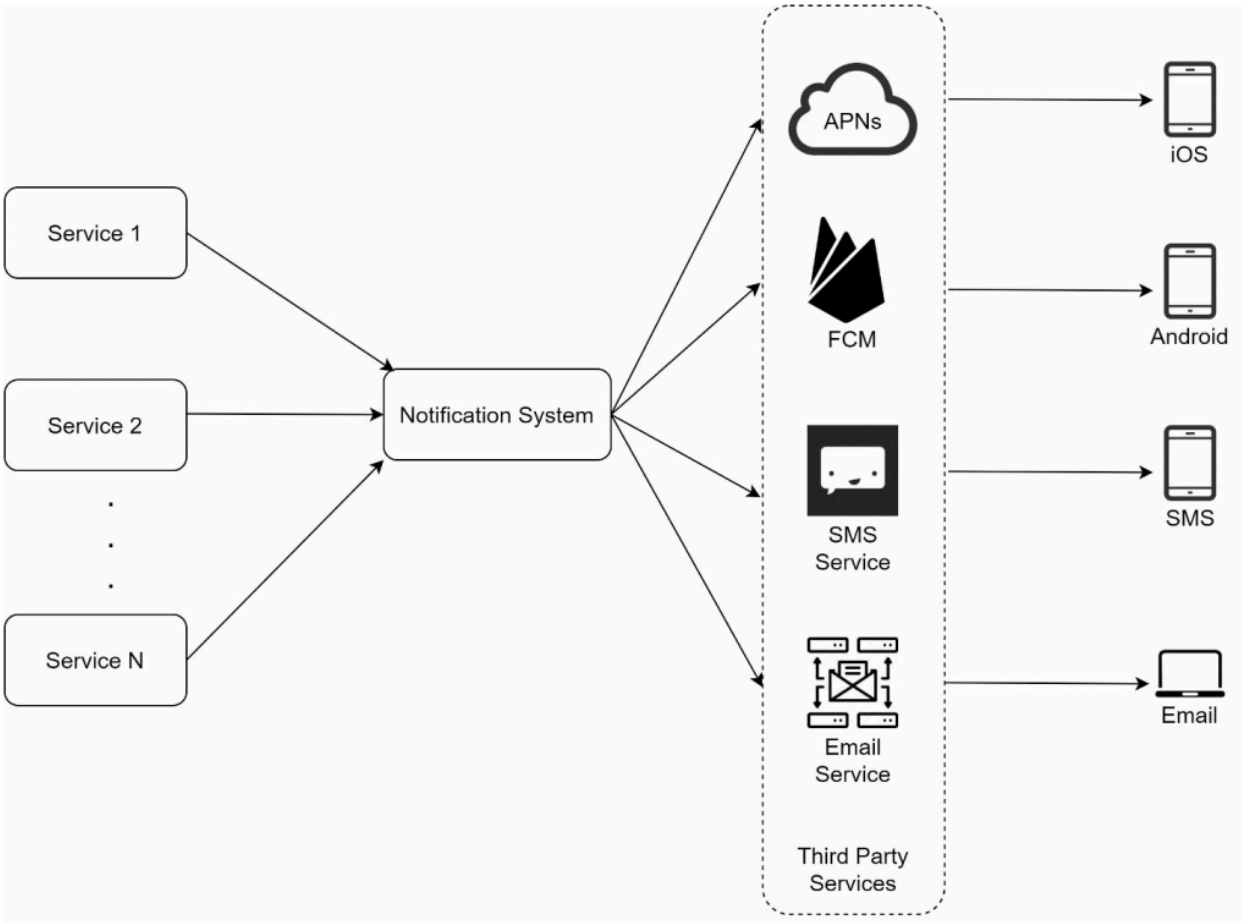
Notification Sending / Receiving Flow

This section first explains the initial high-level design and then identifies its limitations.

High-Level Design



1. Service 1 to N

These services trigger notifications.

Examples:

- Billing service sending payment reminders
- Shopping service sending delivery updates
- Cron jobs sending scheduled notifications

Services can be:

- Microservices
- Cron jobs
- Distributed systems

2. Notification System

The notification system is the core component responsible for:

- Receiving notification requests
- Building notification payloads
- Communicating with third-party services

Initially:

- Only one notification server is used.

Responsibilities:

- Provide APIs
- Build payloads
- Forward notifications

3. Third-Party Services

Third-party services are responsible for actual delivery.

Examples:

- APNS
- FCM
- Twilio

- SendGrid

Important Considerations

Extensibility

The system should allow:

- Plugging new services
- Removing old services easily

Example:

- FCM may not work in China
- Alternatives like Jpush or PushY may be required

4. Client Devices

Users receive notifications on:

- iOS devices
- Android devices
- SMS
- Email

Problems in Initial Design

Three major problems exist in the initial architecture.

1. Single Point of Failure (SPOF)

Only one notification server exists.

If it fails:

- Entire notification system becomes unavailable.

2. Hard to Scale

One server handles:

- Push notifications
- SMS
- Emails
- Databases
- Caches
- Processing

This makes independent scaling difficult.

3. Performance Bottleneck

Notification processing is resource intensive.

Examples:

- Building HTML emails
- Waiting for third-party responses
- Processing large notification batches

During peak load:

- System may become overloaded.